Ops Console (Web, Next.js): Frontend and Backend Handover

Scope and decision: one console or two websites

Decision: one Next.js website with role-based access. Staff sign in once, and the menus and routes they can open depend on their role. The server checks the role on every request; hiding a menu item is not the security.

Why one site

- One login, one deploy, one design system and one set of components.
- Hub leads, admins and support agents share the same order, rider and gate data. Two sites would duplicate the screens or the API calls.
- Permissions are a matter of roles, which the backend already needs for the rest of the system.
- Developer time is limited. One app means one build pipeline and one test suite.

Why not two websites

- A separate admin site would need its own login and its own data calls, for the same data.
- Two sites are worth it only if the admin tools needed a different network zone (for example, a private network only). Legal and security review decide that later.

When to split later

- If the admin tools must sit on a separate network, or if the team grows and the two areas need separate release cycles. Keep route groups clean so a split stays cheap.

What this doc covers

One console with four surfaces: the hub ops panel, the admin console, help and support, and the live ops dashboard. Each surface is a route group in the same Next.js app.

Backend needed before you start

Developer 1 owns the console backend. The console can start on a mock once the contract exists.

1. Console endpoints in the OpenAPI contract

- Listed in the API section below, grouped by surface: /v1/ops (live ops and hub ops), /v1/admin, /v1/support.
- Every response includes server_time.
- Every write that changes state requires an Idempotency-Key header and writes an audit event.
- Role is checked on the server for every endpoint.

2. Mock server (Prism or equivalent)

- Answers every endpoint with schema-valid data.
- Scenario header X-Mock-Scenario selects the state.

- Scenarios the console must handle: zone kill switch on and off; refund above limit (needs second approver); refund approved by second approver; hub timer expired; hub queue empty; shortfall awaiting approval; quarantine awaiting approval; rider verification failed; gate edit conflict (two editors); audit log empty; user without permission (403).

3. Live channel mock

- WebSocket feed for the live map, hub queues and alerts, with server_time on each event.

4. Fixtures

- Test zone, two hubs, staff accounts for each of the six roles, sample orders in each state, sample riders with verification states, and test gates.

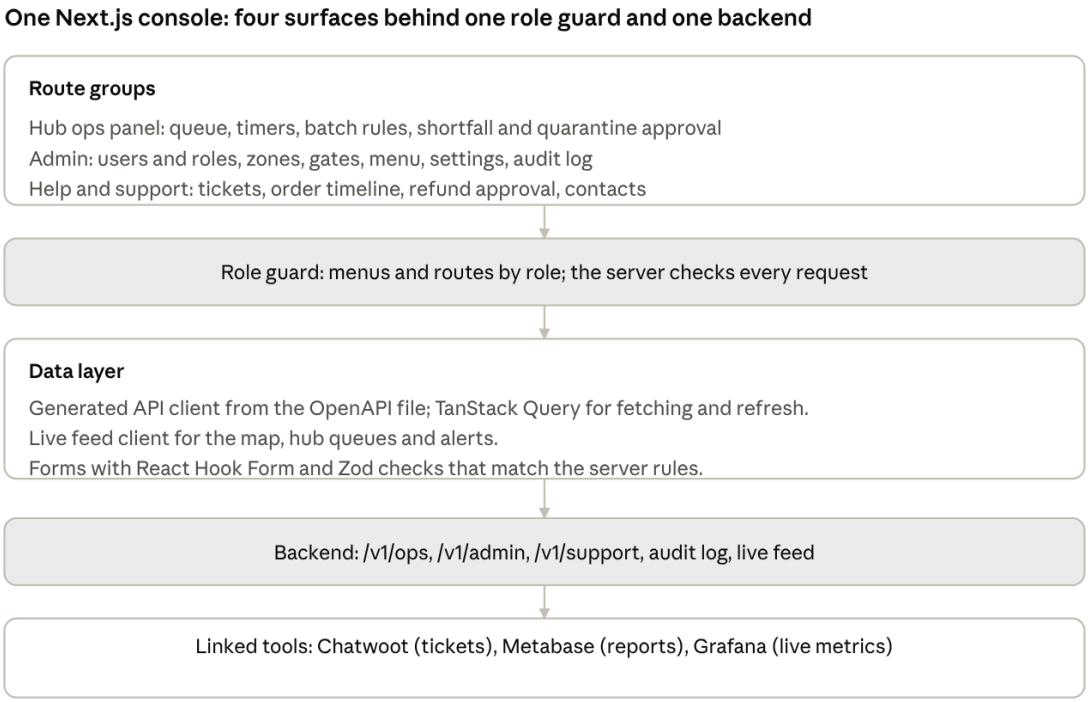
5. Environments

- Mock URL for development.
- Staging URL with test payment keys, from Phase 2.

What to do while waiting

- Build the role guard, the layout and the static screens on the mock.
- Build the live map and hub queue screens on the live-channel mock.

Architecture



ops console architecture · one app, four route groups, one backend

One Next.js app serves four route groups. Every page passes the role guard, every data call passes the server role check, and the reporting and support tools are linked out rather than rebuilt.

Tech stack and libraries

Next.js is decided. Items marked Verify must be checked before use. Frontend and backend listed separately.

Frontend (Next.js app)

Need	Choice	Status
Framework	Next.js (App Router), TypeScript	Decided
UI kit	Tailwind CSS with shadcn/ui (Radix primitives)	Proposed; verify licence and version
Data fetching	TanStack Query	Proposed
Tables	TanStack Table with virtual rows for long lists	Proposed
Forms	React Hook Form with Zod	Proposed
Live feed	WebSocket client, throttled re-render	Proposed
Live map	MapLibre GL JS (scooters, dead zones)	Proposed; verify
Charts (small, in-app)	Recharts	Proposed
Auth (session)	Secure httpOnly cookie session from the backend; no tokens in the browser	Proposed
Crash and error reports	Sentry (browser and server)	Proposed

Backend (Go, same service family as the apps)

Need	Choice	Status
Language and router	Go, chi (as in the backend handover)	Decided
Database access	pgx and sqlc	Decided
Auth	Staff login with password or OTP, plus two-factor (TOTP) for admin and approvers	Proposed; verify
Role checks	A central permission table; checked in middleware for every route	Proposed
Audit log	Append-only table written in the same transaction as the change	Decided
Live feed	WebSocket from the same service	Proposed
Approvals	Two-step approval table for refunds and quarantine above limits	Proposed

Linked tools (not rebuilt)

Need	Choice	Status
Support tickets	Chatwoot, self-hosted in the India region	Recommended; legal to confirm licence
Reports	Metabase on a read-only replica	Recommended; legal to confirm licence
Live metrics	Grafana	Existing plan

Rules

- Pin Node and Next.js versions in the repo.
- Server components only for read-only pages; client components for live screens.
- No direct database access from the frontend; everything goes through the API.

Tools

- Source and CI: GitHub with GitHub Actions; lint, type check, unit tests, end-to-end tests and build on every pull request.
- Unit and component tests: Vitest with React Testing Library.
- End-to-end: Playwright, running each role's main journey against the mock.
- API contract: Schemathesis against the OpenAPI file; Prism mock server.
- Security checks: dependency audit on every build; static analysis (CodeQL or equivalent) on the backend; OWASP ZAP scan on staging before each release.
- Hosting: the India region, with the same deploy path as the backend.
- Error reports: Sentry for the browser and server.
- Design handoff: Figma, read-only; tokens exported as files.
- Task tracking: the team's chosen tracker.

Screens and tasks by surface

Each screen lists its data, its states and its done-when. Every screen checks the role before it renders.

Hub ops panel (hub lead, ops)

- H1. Hub queue: GET /v1/ops/hubs/{id}/restock. Requests with rider arrival time, state and slot. States: empty; late rider; request taken. Done when: a late rider can be moved to another slot and the rider is told.
- H2. Timers: hold timer, packing timer, request expiry and handover window, set in settings. States: timer expired (request reassigned or flagged). Done when: every timer value comes from settings and an expiry follows the stated rule.
- H3. Batch and cassette rules: batch size, cassette capacity, lot selection (soonest expiry first). Data: GET and PUT /v1/ops/hubs/{id}/rules. Done when: a rule change is logged and applies to the next request only.
- H4. Shortfall approval: POST /v1/ops/shortfalls/{id}/approve or reject. Done when: an approved substitute reaches the rider and the kitchen plan.
- H5. Quarantine approval: POST /v1/ops/lots/{id}/quarantine/approve. Done when: an approved quarantine blocks the lot everywhere.
- H6. Hub staff shifts and devices: GET and PUT /v1/ops/hubs/{id}/staff. Done when: a shift change shows on the hub's tablets.
- H7. Hub settings: opening hours, capacity, pause a hub. Done when: a paused hub receives no new requests.

Admin console (admins)

- A1. Users and roles: GET, POST, PUT /v1/admin/users. Done when: a new user sees only the surfaces for their role.
- A2. Zones and kill switch: GET and PUT /v1/admin/zones/{id}/settings (time budget, per-gate caps, 90-second offer on or off). Confirmation and reason required. Done when: a change takes effect and is logged.
- A3. Gate library: GET, POST, PUT /v1/admin/gates; history view. Done when: every edit keeps the old value and who changed it.
- A4. Menu, prices and availability rules: GET and PUT /v1/admin/menu. Done when: a price change is logged and shows in the customer app.
- A5. Rider onboarding and verification queue: GET /v1/admin/riders/review; approve or reject. Done when: a failed DigiLocker case can be approved or sent to the manual path.
- A6. Feature flags: GET and PUT /v1/admin/flags. Done when: each flag change is logged and needs a second admin.
- A7. Audit log viewer: GET /v1/admin/audit?from=&to=&actor=. Read-only. Done when: any ops action can be found by time, person or type.

Help and support (support agents, support leads)

- S1. Ticket list and detail: linked to Chatwoot; order link on each ticket. Done when: a ticket opens the order timeline in one click.
- S2. Order timeline: GET /v1/support/orders/{id}/timeline; every state with timestamp, evidence and actor. Done when: the timeline shows every state with its source.
- S3. Refunds: POST /v1/support/refunds; above the limit, a second approver is needed. Done when: a refund above the limit cannot complete without the second approver.
- S4. Replacement: POST /v1/support/replacements. Done when: a replacement creates one order and links it to the ticket.
- S5. Contact: masked calls to customer or rider through the telephony provider. Done when: no full phone number is shown to support.
- S6. Incident log: POST /v1/support/incidents, linked to order or hub. Done when: an incident shows on the hub's queue and in reports.

Live ops dashboard (dispatchers, ops leads)

- L1. Live map: scooters, orders, dead zones; feed over WebSocket. States: many scooters; stale position; feed dropped. Done when: positions update without a reload.
- L2. Zone controls: kill switch and time budget, pointing to A2. Done when: a zone switch is confirmed before it applies.
- L3. Queues across hubs: GET /v1/ops/hubs/summary. Read-only for dispatchers. Done when: late and short requests show at the top.
- L4. Alerts: dead zone, temperature breach, GPS anomaly. Done when: an alert names the zone and the next action.
- L5. Links out: Metabase reports and Grafana metrics open in a new tab.

Shared

- Sign-in and two-factor for admin and approvers. Done when: a user without two-factor cannot reach admin screens.
- Role-based menus and route guards. Done when: each role's menu matches the role table in the rules section.

API endpoints the console calls

All paths are under /v1. Every request uses the session cookie; every write sends an Idempotency-Key header and writes an audit event. Errors use one shape: code, message, retryable. A missing permission returns 403 with the permission name.

Area	Method and path	Request key fields	Response key fields	Used in
Session	POST /v1/auth/staff/login	email or staff_id, password or OTP	session (cookie), requires_2fa	all
Session	POST /v1/auth/staff/2fa	totp_code	session (cookie), role	all
Hub queue	GET /v1/ops/hubs/{id}/restock	state, slot	requests[] (id, rider_arrival_at, slot, state)	H1
Hub queue	PUT /v1/ops/restock/{id}/slot	slot_id, reason	request (slot)	H1
Timers	GET and PUT /v1/ops/settings/timers	hold_s, packing_s, expiry_s, handover_s	timers	H2
Hub rules	GET and PUT /v1/ops/hubs/{id}/rules	batch_size, cassette_capacity, lot_rule	rules (version)	H3
Shortfall	POST /v1/ops/shortfalls/{id}/approve	decision (approve, reject), reason	shortfall (state)	H4
Quarantine	POST /v1/ops/lots/{id}/quarantine/approve	decision, reason	lot (state)	H5
Hub staff	GET and PUT /v1/ops/hubs/{id}/staff	shifts[], device_id	staff[]	H6
Hub settings	PUT /v1/ops/hubs/{id}/status	open, paused, reason	hub (status)	H7
Users	GET, POST /v1/admin/users; PUT /v1/admin/users/{id}	email, role, active	user (id, role)	A1
Zones	GET and PUT /v1/admin/zones/{id}/settings	time_budget_s, gate_cap, offer_90s_enabled, reason	settings (version)	A2, L2

Area	Method and path	Request key fields	Response key fields	Used in
Gates	GET, POST /v1/admin/gates; PUT /v1/admin/gates/{id}	label, lat, lng, entry_notes, verified	gate (version, history_url)	A3
Menu	GET and PUT /v1/admin/menu/items/{id}	price, availability_rule	item (version)	A4
Riders review	GET /v1/admin/riders/review; POST /v1/admin/riders/{id}/decision	decision, reason	review (state)	A5
Flags	GET and PUT /v1/admin/flags/{key}	enabled, reason	flag (version, second_approver)	A6
Audit	GET /v1/admin/audit	from, to, actor, type	events[] (actor, type, time, before, after)	A7
Tickets	GET /v1/support/tickets; link to Chatwoot ticket_id	status, assignee	tickets[]	S1
Timeline	GET /v1/support/orders/{id}/timeline	none	events[] (state, at, evidence, actor)	S2
Refunds	POST /v1/support/refunds	order_id, amount, reason	refund (state, needs_approval)	S3
Refund approval	POST /v1/support/refunds/{id}/approve	none (second approver)	refund (state)	S3
Replacement	POST /v1/support/replacements	order_id, reason	order (id)	S4
Contact	POST /v1/support/contact	order_id, party (customer or rider)	masked_call_id	S5
Incidents	POST /v1/support/incidents	type, order_id or hub_id, note	incident (id)	S6
Hub summary	GET /v1/ops/hubs/summary	zone_id	hubs[] (id, late, short, queue)	L3
Live feed	WS /v1/ops/stream?zone_id=	session cookie	map_update, queue_update, alert, server_time	L1, L4

Area	Method and path	Request key fields	Response key fields	Used in
Zone map	GET /v1/ops/zones/{id}/map	none	scooters[], orders[], dead_zones[]	L1

Notes

- The 403 response names the missing permission, so the console can show a clear message instead of a blank screen.
- Any write that needs a second approver returns needs_approval; the approver calls the approve endpoint.
- The live feed sends server_time so the console can show the age of each update.
- Field names are a starting point; agree final names in the first contract review with Developer 2 and Developer 3, since the same data appears in their apps.

Roles, approvals, audit and security

These rules decide who can do what. The server enforces them; the screens only reflect them.

Roles and surfaces

Role	Surfaces it can open	Cannot do
Dispatcher	Live ops (view and zone controls with confirmation)	Change settings, refunds, riders
Hub lead	Hub ops panel for own hub; approve shortfall and quarantine	Change global settings; other hubs
Hub staff	Tablet app only; no console access	Everything in the console
Kitchen	Hub queue summary (read); kitchen batch view	Settings, refunds
Support agent	Help and support; order timeline; small refunds	Approve refunds above limit; zones
Support lead	All support; approve refunds above limit	Zones and users
Admin	Admin console; users; gates; menu; flags (with second approver)	Refund approval (separate role)
Finance	Reports links; refund totals (read)	Any write outside finance views

Rules

1. **Server-side checks.** Every endpoint checks the role and returns 403 with the permission name.
2. **Two-factor for admin and approvers.** TOTP required for admin, support leads and anyone who approves.
3. **Second approver.** Refunds above the limit, quarantine changes, flag changes and kill-switch changes need a second person with the right role.

4. **Audit everything.** Every write records actor, time, what changed (before and after) and the reason, in the same transaction.
5. **Reason required.** Kill switch, flags, refunds, quarantine and gate edits need a short reason.
6. **Masked contact.** No full customer or rider phone number is shown to any role except where the law requires it; calls go through masked numbers.
7. **No secrets in the browser.** Sessions use httpOnly cookies; no API keys in client code.
8. **Least data.** Pages load only the fields their role needs; exports are logged.
9. **Timed session.** Idle sessions time out; admin sessions time out sooner.
10. **Personal data requests.** Export and deletion for a person's data are admin actions, logged, and follow the legal retention rules.

Security checks before release

- Role matrix tests: every endpoint tested with every role.
- Dependency audit on every build; high-severity findings block release.
- OWASP-style scan on staging before each release.

Testing and release checklist

No release goes to live staff until every item is ticked.

- ☐ Role matrix: every endpoint returns 403 for every role that should not have access, and the right data for the roles that should.
- ☐ Two-factor: admin and approver screens are blocked without it.
- ☐ Second approver: a refund above the limit cannot complete with one person.
- ☐ Kill switch: a zone change needs confirmation and a reason; the change is in the audit log.
- ☐ Audit: every write creates one audit event in the same transaction, with before and after values.
- ☐ Hub timers: each timer expiry follows its rule on the mock and on staging.
- ☐ Live map: positions update without a reload; a dropped feed shows a warning and reconnects.
- ☐ Stale data: every screen shows the age of its data.
- ☐ Masked contact: no full phone number appears on any screen or in any export.
- ☐ End-to-end journeys pass for each role's main task (Playwright).
- ☐ Contract tests pass against the Developer 1 OpenAPI file.
- ☐ Dependency audit has no high-severity findings.
- ☐ Security scan on staging has no open high-severity findings.
- ☐ Responsive check at laptop, large monitor and tablet widths.
- ☐ Keyboard use works for every action on the support and hub screens.
- ☐ Staging used by the ops team for one full day before production.

Open points

These are not decided yet. The developer can make the choice listed, record it here, and tell the other developers if it changes an endpoint.

- Developer assignment for the console: not yet decided. Options recorded in the earlier plan: Developer 2 after the customer app, a contractor, or after the pilot.
- Chatwoot and Metabase: legal to confirm the licence and whether self-hosting in the India region is allowed.
- Staff sign-in method: password plus TOTP, or OTP. Confirm with the backend lead.

- Refund limit amount and the second-approver rule: product and finance decide the number.
- Quarantine rule: whether a hub lead can quarantine alone or needs a second approver.
- Kill switch default values for time budget and gate cap: set during the simulator runs.
- Telephony provider for masked calls: not chosen.
- Map provider details for the live map, once the pilot map extract exists.
- Whether the console needs a separate network zone later (see the scope decision).

If a feature or flow changes during the build, record the change in this doc with the date, the reason, and the developer who made it.